

Docker Volumes Configuration

Questa directory contiene tutte le configurazioni persistenti per i servizi dell'infrastruttura del gioco HTML5 singleplayer. Ogni subdirectory corrisponde a un servizio specifico con le sue configurazioni personalizzabili.

Struttura Volumi

Proxy & SSL Management

- `npm_data/` - NGINX Proxy Manager configuration
- `npm_letsencrypt/` - SSL certificates storage

Monitoring & Logging

- `elastic-agent/` - System monitoring configuration
- `fluent-bit/` - Log aggregation rules
- `fluent-bit-db/` - Log processing databases
- `kibana/` - Dashboard configurations
- `logs/` - Centralized log storage
- **APM Configuration** - Application Performance Monitoring setup

Database Storage

- `mongodb/` - Game data persistence

Proxy & SSL Management

`npm_data/`

Scopo: Configurazioni NGINX Proxy Manager per routing e SSL

Struttura:

```
npm_data/
├── database.sqlite      # Configurazioni UI e domini
├── keys.json           # Chiavi API e certificati
├── nginx/              # Configurazioni NGINX
│   ├── proxy_host/    # Virtual hosts configurati
│   │   ├── 4.conf     # localhost → server:3000
│   │   ├── 5.conf     # kibana.localhost → kibana:5601
│   │   ├── 6.conf     # redis-ui.localhost → redis-ui:8081
│   │   ├── 7.conf     # mongo-ui.localhost → mongo-ui:8081
│   │   └── 8.conf     # apm.localhost → apm-server:8200
│   ├── default_host/  # Default configurations
│   ├── dead_host/     # Disabled hosts
│   └── redirection_host/ # URL redirects
└── logs/              # Access e error logs
```

Configurazioni Attuali:

- **Game Frontend:** `localhost` → `server:3000`
- **Kibana Dashboard:** `kibana.localhost` → `kibana:5601`
- **Redis UI:** `redis-ui.localhost` → `redis-ui:8081`
- **MongoDB UI:** `mongo-ui.localhost` → `mongo-ui:8081`
- **APM Server:** `apm.localhost` → `apm-server:8200`

Environment Dependencies:

```
PROXY_ADMIN_PORT=18081          # Admin UI port
DB_SQLITE_FILE=/data/database.sqlite
```

Personalizzazione:

1. **Nuovi domini:** Aggiungi tramite UI su `http://localhost:18081`
2. **SSL certificates:** Configurazione automatica Let's Encrypt
3. **Load balancing:** Configurabile per multiple istanze
4. **Security headers:** Customizzabili per sicurezza gaming

`npm_letsencrypt/`

Scopo: Storage certificati SSL automatici

Contenuto:

- Certificati Let's Encrypt per domini configurati
- Chiavi private e certificati intermedi
- Renewal automatico configurazioni

Backup: Essenziale per produzione - contiene certificati validi

Monitoring & Logging

`elastic-agent/`

Scopo: Monitoring sistema e metriche infrastruttura

File Principale: `elastic-agent.yml`

Configurazione:

```
agent:
  id: gaming-infrastructure-agent
  monitoring:
    enabled: true
```

```
    use_output: default
    namespace: default

outputs:
  default:
    type: elasticsearch
    hosts: ["${ELASTICSEARCH_HOSTS}"]
    username: "${ELASTICSEARCH_USERNAME}"
    password: "${ELASTICSEARCH_PASSWORD}"

inputs:
  - type: system/metrics
    id: system-metrics-gaming
    data_stream:
      namespace: gaming
    processors:
      - add_fields:
          target: gaming
          fields:
            environment: development
            service_type: infrastructure
    streams:
      - metricset: cpu
        period: 10s
      - metricset: memory
        period: 10s
      - metricset: network
        period: 30s
      - metricset: filesystem
        period: 60s
```

Environment Dependencies:

```
ELASTIC_AGENT_STANDALONE=true
ELASTICSEARCH_HOSTS=http://elasticsearch:9200
ELASTICSEARCH_USERNAME=elastic
ELASTICSEARCH_PASSWORD=m60HmMuiqNrV1i25Jz3Z
```

Metriche Raccolte:

- **CPU:** Utilizzo per gaming performance
- **Memory:** RAM usage per cache optimization
- **Network:** Latency e throughput
- **Disk:** Storage per asset e database

Personalizzazione:

1. **Nuovi input:** Aggiungi monitoring per servizi custom
2. **Retention:** Configura data retention per analytics
3. **Alerting:** Setup alert per performance critiche

fluent-bit/

Scopo: Aggregazione e parsing log da tutti i container

File Configurazione:

- **fluent-bit.conf** - Configurazione principale
- **parsers.conf** - Parser per diversi formati log
- **container_name.lua** - Script per identificazione container

Configurazione Principale (**fluent-bit.conf**):

```
[SERVICE]
    Flush                5
    Log_Level            info
    Parsers_File          parsers.conf

# INPUT: Docker containers con metadata servizio
[INPUT]
    Name                 tail
    Path                 /var/lib/docker/containers/*//*.log
    Parser               docker
    Tag                  docker.*

# FILTER: Estrazione nome container e servizio
[FILTER]
    Name                 lua
    Match                docker.*
    Script               /fluent-bit/etc/container_name.lua
    Call                 extract_container_info

# OUTPUT: Invio a Elasticsearch con indici per servizio
[OUTPUT]
    Name                 es
    Match                docker.*
    Host                 elasticsearch
    Port                 9200
    HTTP_User            ${ELASTICSEARCH_USERNAME}
    HTTP_Passwd          ${ELASTICSEARCH_PASSWORD}
    Index                docker_logs
    Suppress_Type_Name   On
```

Parser Configurati (**parsers.conf**):

```
# Parser per log Docker JSON
[PARSER]
    Name    docker
    Format  json
    Time_Key time
```

```

Time_Format %Y-%m-%dT%H:%M:%S.%L

# Parser per log MongoDB
[PARSER]
    Name      mongodb
    Format     regex
    Regex      ^(?<time>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d+)?
(?:Z| [+ -]\d{2}:\d{2}))\s+(?<message>.*)
    Time_Key   time
    Time_Format %Y-%m-%dT%H:%M:%S.%L%z

# Parser per log NGINX Proxy
[PARSER]
    Name      nginx_proxy_error
    Format     regex
    Regex      ^(?<time>\d{4}/\d{2}/\d{2} \d{2}:\d{2}:\d{2}) \[(?
<level>\w+)\] \d+#\d+: \*?\d* ?(?<message>.*)
    Time_Key   time
    Time_Format %Y/%m/%d %H:%M:%S

```

Script Container Detection (`container_name.lua`):

```

function extract_container_info(tag, timestamp, record)
    -- Identifica servizio dal contenuto log
    local service_name = "unknown"
    if record["log"] then
        local log_content = record["log"]

        if string.find(log_content, "elasticsearch") then
            service_name = "elasticsearch"
        elseif string.find(log_content, "kibana") then
            service_name = "kibana"
        elseif string.find(log_content, "mongo") then
            service_name = "mongodb"
        elseif string.find(log_content, "redis") then
            service_name = "redis"
        elseif string.find(log_content, "gin") or string.find(log_content,
"go") then
            service_name = "server"
        elseif string.find(log_content, "vite") then
            service_name = "frontend"
        end
    end

    record["container_name"] = service_name
    record["service_name"] = service_name

    return 1, timestamp, record
end

```

Environment Dependencies:

```
ELASTICSEARCH_USERNAME=elastic
ELASTICSEARCH_PASSWORD=m60HmMuiqNrV1i25Jz3Z
```

Personalizzazione:

1. **Nuovi Parser:** Aggiungi parser per nuovi formati log
2. **Filtri Custom:** Crea filtri per eventi specifici gioco
3. **Output Multipli:** Configura invio a sistemi esterni
4. **Retention:** Configura retention policy per log

fluent-bit-db/

Scopo: Database per tracking stato log processing

File:

- `flb_container_logs.db` - Stato processing log container
- `flb_proxy_error.db` - Stato processing log proxy
- `flb_mongodb.db` - Stato processing log database

Funzione: Evita duplicazione log dopo restart servizi

kibana/

Scopo: Configurazioni dashboard e data visualization

Struttura:

```
kibana/
├── config/
│   └── kibana.yml          # Configurazione principale
└── data/                  # Dashboard e index patterns salvati
```

Configurazione Principale (`config/kibana.yml`):

```
# Connessione Elasticsearch
elasticsearch:
  hosts: [ "http://elasticsearch:9200" ]
  username: "kibana_system"
  password: "eJL-r*4rNjRio0wWPEMT"

# Server configuration
server:
  name: kibana
```

```
host: "0.0.0.0"
ssl:
  enabled: false

# Security e encryption
xpack:
  encryptedSavedObjects:
    encryptionKey: "anotherExtremelyExtremelySecureKeyThatIsLong!"

# APM Integration
apm:
  ui:
    enabled: true
    serviceMapEnabled: true

# Fleet Management
fleet:
  agents:
    enabled: true
  packages:
    - name: apm
      version: latest
```

Environment Dependencies:

```
ELASTICSEARCH_HOSTS=http://elasticsearch:9200
KIBANA_SYSTEM_PASSWORD=eJL-r*4rNjRio0wWPEMT
APM_SERVER_URL=http://apm.localhost
```

Dashboard Gaming:

1. **Game Performance:** Metriche APM per gameplay
2. **Player Analytics:** Comportamento utenti
3. **Infrastructure Health:** Status servizi
4. **Error Tracking:** Monitoring errori applicazione

Personalizzazione:

1. **Index Patterns:** Configura pattern per nuovi indici
2. **Visualizations:** Crea grafici custom per gaming metrics
3. **Alerts:** Setup alerting per eventi critici
4. **Spaces:** Organizza dashboard per team diversi

logs/

Scopo: Storage centralizzato per log di sistema

Struttura:

```
logs/
├── esdata/           # Elasticsearch data directory
├── mongodb/         # MongoDB log files
└── (altri log sistema)
```

Retention: Configurabile tramite Elasticsearch ILM policies

APM Configuration

Application Performance Monitoring Setup

Scopo: Monitoring performance applicazioni con token separati per sicurezza

Environment Variables Overview

Core APM Configuration:

```
# Main APM Server configuration
APM_SECRET_TOKEN=apm-secret-token-123
APM_SERVER_URL=http://apm.localhost
APM_SERVER_INTERNAL_URL=http://apm-server:8200

# Elasticsearch connection
ELASTICSEARCH_HOSTS=http://elasticsearch:9200
ELASTICSEARCH_USERNAME=elastic
ELASTICSEARCH_PASSWORD=m60HmMuiqNrV1i25Jz3Z
```

Backend APM (Go Server):

```
# APM agent configuration for Go backend
ELASTIC_APM_SERVER_URL=http://apm.localhost
ELASTIC_APM_SERVICE_NAME=go-backend
ELASTIC_APM_ENVIRONMENT=development
ELASTIC_APM_SECRET_TOKEN=${APM_SECRET_TOKEN} # References main token
```

Frontend RUM (JavaScript):

```
# Real User Monitoring for frontend
ELASTIC_APM_RUM_SERVER_URL=http://apm.localhost
ELASTIC_APM_RUM_SERVICE_NAME=frontend-app
ELASTIC_APM_RUM_SECRET_TOKEN=rum-secret-token-456 # Different token for RUM
```

NPM Registry Access:

```
# NPM Registry credentials for private packages
NPM_EMAIL=dev.progettiinnovativi.2025@gmail.com
NPM_PASSWORD=MmkjJTaz@GTn8p
```

Secret Token Strategy

Why Different Tokens?:

1. **Security Separation:** Backend e frontend hanno contesti di sicurezza diversi
2. **Permission Granularity:** Agenti diversi possono necessitare permessi diversi
3. **Token Rotation:** Possibilità di ruotare token indipendentemente per tipo servizio
4. **Monitoring:** Tracking più facile per identificare tipo agente che invia dati

Token Configuration:

Backend Token (Server-side):

- **Token:** `apm-secret-token-123`
- **Used by:** Go backend service
- **Monitors:** Database queries, HTTP requests, external API calls
- **Environment Variable:** `ELASTIC_APM_SECRET_TOKEN`

RUM Token (Client-side):

- **Token:** `rum-secret-token-456`
- **Used by:** Frontend JavaScript application
- **Monitors:** Page loads, user interactions, browser performance
- **Environment Variable:** `ELASTIC_APM_RUM_SECRET_TOKEN`

Docker Compose Integration

Elastic Agent Configuration:

```
elastic-agent:
  environment:
    - ELASTICSEARCH_HOSTS=${ELASTICSEARCH_HOSTS}
    - ELASTICSEARCH_USERNAME=${ELASTICSEARCH_USERNAME}
    - ELASTICSEARCH_PASSWORD=${ELASTICSEARCH_PASSWORD}
```

Backend Server Configuration:

```
server:
  environment:
    - ELASTIC_APM_SERVER_URL=${ELASTIC_APM_SERVER_URL}
    - ELASTIC_APM_SERVICE_NAME=${ELASTIC_APM_SERVICE_NAME}
    - ELASTIC_APM_ENVIRONMENT=${ELASTIC_APM_ENVIRONMENT}
    - ELASTIC_APM_SECRET_TOKEN=${ELASTIC_APM_SECRET_TOKEN}
```

Frontend Configuration:

```
frontend:
  environment:
    - VITE_ELASTIC_APM_RUM_SERVER_URL=${ELASTIC_APM_RUM_SERVER_URL}
    - VITE_ELASTIC_APM_RUM_SERVICE_NAME=${ELASTIC_APM_RUM_SERVICE_NAME}
    - VITE_ELASTIC_APM_RUM_SECRET_TOKEN=${ELASTIC_APM_RUM_SECRET_TOKEN}
    - VITE_ELASTIC_APM_ENVIRONMENT=${ELASTIC_APM_ENVIRONMENT}
```

Configuration Files Updated

Elastic Agent (`elastic-agent.yml`):

```
# Environment variable usage for centralized configuration
hosts:
  - '${ELASTICSEARCH_HOSTS}'
username: '${ELASTICSEARCH_USERNAME}'
password: '${ELASTICSEARCH_PASSWORD}'
```

Frontend RUM Configuration:

- File: `frontend/src/apm-rum-config.js`
- Utilizzo environment variables per inizializzazione RUM agent

Adding Additional Agents

For New Services:

1. Create new secret token:

```
# In .env file
ELASTIC_APM_NEWSERVICE_SECRET_TOKEN=newservice-token-789
ELASTIC_APM_NEWSERVICE_SERVICE_NAME=my-new-service
```

2. Update `docker-compose.yml`:

```
newservice:
  environment:
    - ELASTIC_APM_SECRET_TOKEN=${ELASTIC_APM_NEWSERVICE_SECRET_TOKEN}
    - ELASTIC_APM_SERVICE_NAME=${ELASTIC_APM_NEWSERVICE_SERVICE_NAME}
```

For Different Agent Types:

- **Mobile Apps:** Token separato `mobile-app-token-abc`
- **Microservices:** Token service-specific `auth-service-token-def`
- **Third-party Integrations:** Token integration-specific

Verification Commands

Check Environment Variables:

```
# Check Elastic Agent variables
docker exec elastic-agent env | grep -E "(ELASTICSEARCH|ELASTIC_APM)"

# Check Go server APM variables
docker exec server env | grep -E "ELASTIC_APM"

# Check frontend variables (during build)
docker exec frontend env | grep -E "VITE_ELASTIC_APM"
```

Test APM Connectivity:

```
# Generate test traffic
docker exec elasticsearch bash -c 'for i in {1..10}; do curl -s
http://server:3000/health; done'

# Check APM traces
docker exec elasticsearch curl -s -u "elastic:password"
"http://localhost:9200/traces-apm*/_count"
```

APM Best Practices

1. **Use descriptive token names** che indicano il loro scopo
2. **Rotate tokens regularly** per sicurezza
3. **Use different tokens per environment** (dev/staging/prod)
4. **Monitor token usage** in APM Server logs
5. **Document token permissions** e scope di utilizzo

APM Troubleshooting

Common Issues:

1. **403 Forbidden:** Verifica che secret token corrisponda alla configurazione APM Server
2. **Connection Refused:** Verifica che APM Server URL sia corretto per networking container
3. **No Data in Kibana:** Controlla configurazione agent e permessi token
4. **Token Mismatch:** Assicurati che frontend usi RUM token, backend usi server token

Debug Commands:

```
# Check APM Server logs
docker logs apm-server --tail 20

# Check agent logs
docker logs server --tail 20

# Verify APM indices
docker exec elasticsearch curl -s -u "elastic:password"
"http://localhost:9200/_cat/indices/apm-*
```

Database Storage

mongodb/

Scopo: Persistenza dati di gioco

Contenuto:

- Database giocatori e profili
- Leaderboard e achievement
- Session data e progressi
- Configurazioni di gioco

Struttura File:

```
mongodb/
├── _mdb_catalog.wt          # Catalog metadata
├── collection-*.wt         # Game data collections
├── index-*.wt              # Database indexes
├── journal/                # Transaction logs
└── diagnostic.data/        # Performance diagnostics
```

Environment Dependencies:

```
MONGODB_HOST=db
MONGO_URI=mongodb://db:27017/database
```

Backup Strategy:

```
# Backup automatico daily
docker exec db mongodump --out /data/backup/$(date +%Y%m%d)

# Restore da backup
docker exec db mongorestore /data/backup/20250730
```

Collections Gaming:

- **players** - Profili giocatore e preferenze
 - **progress** - Progressione livelli e achievement sbloccati
 - **sessions** - Sessioni di gioco attive e cronologia
 - **scores** - Migliori punteggi per livello
 - **game_config** - Configurazioni globali e bilanciamento
 - **analytics_events** - Eventi di gameplay per analisi comportamentali
-

Configurazione Avanzata & Best Practices

⚠ Per le complete best practices di sviluppo e configurazione consultare: 

[GAME_ARCHITECTURE.md](#)

Quick Configuration Guide

Aggiungere Nuovo Servizio Monitoring:

1. **Elastic Agent**: Aggiungi input in `elastic-agent/elastic-agent.yml`
2. **Fluent Bit**: Crea parser in `fluent-bit/parsers.conf`
3. **Kibana**: Configura dashboard specifico
4. **Environment**: Aggiungi variabili necessarie in `.env`

Configurazione Multi-Environment:

```
# Development
cp docker/volumes/kibana/config/kibana.dev.yml
docker/volumes/kibana/config/kibana.yml

# Production
cp docker/volumes/kibana/config/kibana.prod.yml
docker/volumes/kibana/config/kibana.yml
```

Security Hardening:

- **Encryption**: Aggiorna encryption keys in configurazioni
- **Access Control**: Configura RBAC per servizi
- **Network Policies**: Restrizioni traffico tra servizi
- **Audit Logging**: Abilitazione audit trail

Performance Optimization:

- **Index Templates**: Ottimizza mapping Elasticsearch
 - **Retention Policies**: Configura ILM per storage efficiency
 - **Cache Configuration**: Ottimizza cache Redis per gaming
 - **Resource Limits**: Configura limiti CPU/RAM per container
-

Troubleshooting

Problemi Comuni

Elasticsearch non si avvia:

```
# Check disponibilità memoria
free -h
# Minimum 2GB required

# Check permissions
sudo chown -R 1000:1000 docker/volumes/logs/esdata
```

Fluent Bit non invia log:

```
# Check configurazione
docker logs fluent-bit --tail 20

# Test connettività Elasticsearch
docker exec fluent-bit curl -u elastic:password
http://elasticsearch:9200/_cluster/health
```

Kibana non carica dashboard:

```
# Check index patterns
curl -u elastic:password http://localhost:9200/_cat/indices

# Recreate index pattern
# Via Kibana UI: Stack Management → Index Patterns
```

Monitoring Health

```
# Check tutti i volumi
./scripts/check-persistent-config.sh

# Verifica singolo servizio
docker-compose -f docker-compose.dev.yml ps <service_name>

# Log specifici
docker logs <container_name> --tail 50 -f
```